

SEMANTIC WEB PROJECT

VN-Edu LOD

A Knowledge Graph of Vietnamese Higher-Education Institutions

Design, Implementation, and Evidence-based Evaluation

Revised English project report

Ontology 2.2 and integration edition: 8 October 2026

This report presents the design and implementation of a Linked Open Data knowledge graph for Vietnamese higher-education institutions. It combines a domain ontology, a reproducible multi-source transformation pipeline, RDF publication, reasoning, validation, and SPARQL access. The report is based on the corrected repository release and reports measured results with their scope, rather than treating a successful query or a visualization as proof of real-world truth.

Project repository:

<https://github.com/pham-ng/Vietnam-University-Knowledge-Graph-ver2>

Abstract

VN-Edu LOD is an RDF knowledge graph for Vietnamese educational institutions. The project integrates structured Wikidata records, Vietnamese Wikipedia material, and curated reference tables into a common model of organizations, people, academic programs, and administrative geography. The corrected snapshot contains 300 educational organizations, 1,702 people, 34 current and 29 former province-level units, and 78 illustrative programs. Its central technical contribution is a staged Bronze–Silver–Gold pipeline: source snapshots are normalized into records, records are mapped to persistent HTTP IRIs, RDF is linked and materialized, and the resulting union is released only after structural and semantic checks.

The ontology contains 39 local classes, 38 object properties, and 27 datatype properties. OWL 2 RL profile checking, rule-based materialization, SHACL validation, Apache Jena Fuseki/TDB2 acceptance tests, and regression tests provide complementary evidence. The validated serving union contains 73,628 distinct triples. The report separates ontology conformance, reference-link agreement, data completeness, factual accuracy, and production readiness: they are related engineering goals, but none is a substitute for the others. The result is a reproducible educational Linked Data artifact with an explicit list of remaining evidence requirements for authoritative, high-load deployment.

Keywords: knowledge graph; RDF; ontology; SHACL; SPARQL; entity resolution; data provenance; Vietnamese higher education.

Contents

1	Project overview	6
1.1	Motivation and problem statement	6
1.2	Objectives and contributions	6
1.3	Research and evaluation questions	7
2	Semantic Web foundations and requirements	7
2.1	Standards used by the project	7
3	Data sources and collection	8
3.1	Source roles	8
3.2	Scope and selection effects	9
4	Architecture and data management	9
4.1	Layers and contracts	9
4.2	Identity and URI persistence	10
4.3	Provenance and disputed values	10
5	System and software architecture	11
5.1	Component responsibilities	11
5.2	Technology choices	12
5.3	Tooling stack	12
6	Data transformation and knowledge-graph population	13
6.1	Bronze layer: collection and source preservation	13
6.2	Silver layer: integration and normalization	13
6.3	Gold layer: RDF transformation, linking and reasoning	14
6.4	Concrete Silver-to-RDF mapping	14
6.5	Release boundary	14
7	Ontology design and relationship semantics	15
7.1	Design method	15
7.2	Modules	15
7.3	Ontology overview	16
7.4	TBox structure and property semantics	16
7.5	Protégé inspection and ABox population	18
7.6	Reasoning and conformance	18
7.7	Materialization and temporal interpretation	20
8	Linking and publication	20

8.1	External relations	20
8.2	Representations and deployment	21
8.3	User-facing resource and query views	21
9	Deployment and Apache Jena Fuseki	24
9.1	Identity, representation and storage	24
9.2	Safe loading and reproducible runtime	24
9.3	Production-like security and performance acceptance	25
9.4	Production hardening profile and release gates	25
9.5	Executed end-to-end acceptance evidence	26
10	Five-star Linked Open Data assessment	27
10.1	Dated HTTP evidence	27
11	Application security and query execution	28
12	Reproducible workflow and CI/CD	29
12.1	Release sequence	29
12.2	What CI proves	29
13	Evaluation and results	30
13.1	Release measurements	30
13.2	Link experiment: agreement, not independent accuracy	30
13.3	Executed Silk Framework experiment	31
13.4	Regression evidence	32
13.5	Treatment of earlier comparisons	32
14	Discussion, limitations and future work	32
15	Conclusion	33
A	Reproducing the release and report	33
B	A competency-query example	35

List of Figures

1	Validated release flow	10
2	Overall VN-Edu LOD architecture	11
3	Software architecture and repository components	11
4	VN-Edu technology stack	12
5	Data transformation workflow	13
6	Overview of the VN-Edu ontology	16
7	TBox class hierarchy and property signatures	17
8	Protégé inspection of the ontology	19
9	ABox instance neighborhood	19
10	Generated Linked Data resource view for a university.	22
11	Read-only SPARQL query view and returned result table.	23
12	URI publication and optional Fuseki execution	24
13	CI/CD and evidence workflow	29

List of Tables

1	Project contributions and evidence.	6
2	Design requirements and the mechanism used to address them.	8
3	Source roles and integration treatment.	8
4	Main software components and their boundaries.	13
5	Selected Silver-to-RDF mappings for one institution record.	14
6	Main modules and intended distinctions.	15
7	TBox, ABox and validation responsibilities.	18
8	Technology roles in the VN-Edu publication architecture.	21
9	Production release gates that remain separate from OWL and SHACL conformance. . .	26
10	Measured Fuseki/TDB2 acceptance results.	27
11	Evidence against the cumulative five-star publication scheme.	27
12	Observed HTTP behavior; deployed content may predate the corrected branch.	28
13	Measured size of the corrected local release.	30
14	Token-Jaccard agreement with the identifier-derived reference.	30
15	Actual Silk versus the fixed-sample Python baseline: reference agreement only.	31

1 Project overview

1.1 Motivation and problem statement

Information about a university is distributed across knowledge bases, encyclopedic articles, institutional pages, and administrative reference data. The same institution may have several names, identifiers, locations, or organizational relationships. These differences matter in practice: a member school is not necessarily a branch campus, direct governance is not the same as state oversight, and a former province is not interchangeable with its successor. An RDF knowledge graph is useful here because it represents those distinctions as typed, linkable statements that can be queried across sources.

VN-Edu LOD is an engineering case study of this integration problem. It is designed to demonstrate a complete Semantic Web workflow, from source collection and URI assignment to ontology-based transformation, reasoning, validation, publication, and query execution. It is not presented as a complete official register of Vietnamese higher education. Names, classifications, dates, and relationships describe the collected snapshot and the project’s documented extraction policies.

1.2 Objectives and contributions

The project has four practical objectives:

1. build a domain ontology that distinguishes organization, governance, people, education, geography, identity, and temporal evidence;
2. transform heterogeneous records into persistent RDF resources and publish machine-readable representations using Linked Data principles;
3. evaluate the release with OWL 2 RL profile checking, materialization, SHACL, SPARQL, and automated regression tests; and
4. demonstrate reproducible execution with Apache Jena Fuseki and a controlled Silk candidate-linking experiment.

The resulting contributions are summarized in Table 1. This framing follows a project-report structure: each contribution is paired with an artifact and an evidence source, so that the report can distinguish implementation from evaluation.

Table 1: Project contributions and evidence.

Contribution	Implemented artifact	Evidence reported in this document
Domain model	OWL/Turtle ontology 2.2 with classes, properties, signatures, alignments, and selected chains	Protégé inspection, OWL 2 RL profile gate, ontology figures, and reasoning examples
Data integration	Bronze–Silver–Gold transformation pipeline, URI registry, provenance and conflict logs	Reproducible pipeline, mapping example, release fingerprints, and coverage counts
Linked publication	HTML, Turtle, JSON-LD, stable resource IRIs, link graph, and metadata	Publication checks, five-star assessment, and explicit licensing/identity scope
Operational checks	Flask read-only query path, Fuseki/TDB2 loader, CI workflows, SHACL and regression suite	End-to-end acceptance tables, security probes, and test results

1.3 Research and evaluation questions

The evaluation is organized around four questions:

1. Does the model distinguish the relationships needed for institutional, educational, geographic, and governance queries without producing detected local contradictions?
2. Can the transformation reproduce a validated release while preserving resource identity when labels or input order change?
3. What do the measurements establish about structure, coverage, link agreement, and factual correctness, and which claims remain unsupported?
4. Can publication and query handling fail safely on stale releases, invalid input, unapproved federation, and bounded overload?

The review covers the source code, ontology, SHACL shapes, cached inputs, generated RDF, tests, CI workflows, and earlier project documentation. Security counterexamples are executed locally; external production endpoints are not used as attack targets.

2 Semantic Web foundations and requirements

2.1 Standards used by the project

RDF describes resources through triples. Subjects may be IRIs or blank nodes; predicates are IRIs, and objects may additionally be literals. A graph is a set, so duplicate triples do not increase its size. This matters when combining asserted data, ontology statements, links, and inferred statements: their file counts cannot simply be added to obtain the size of the union [1].

OWL axioms express implications about the modeled world. A property's domain is not an input-validation rule: using the property can entail that its subject belongs to that domain. Similarly, a functional object property can imply equality between two values; it does not necessarily reject the second value. The project therefore supplements reasoning with explicit validation of asserted records. The OWL 2 profile specification defines restrictions under which particular reasoning guarantees apply [2].

SHACL validates a supplied data graph against declared constraints [3]. Conformance is relative to those constraints, not a claim that every statement is true. Here, missing values for some descriptive fields generate warnings, while invalid datatypes, selected cardinality violations, and hard constraints prevent publication.

SPARQL supplies query syntax and algebra for RDF, including optional patterns, aggregates, and remote SERVICE clauses [4]. Exposing it over HTTP introduces an execution boundary: query syntax, remote destinations, elapsed work, and publication state must all be controlled independently of the user interface.

The project reuses identifiers and vocabulary from Wikidata, DBpedia, schema.org, FOAF, SKOS, and PROV-O. PROV-O supports descriptions of derivation [5]; whether the provenance is attached to an entity, a graph, or an individual assertion determines what can actually be traced. HTTP identifiers and useful representations support Linked Data publication, while persistence requires more than generating readable slugs [6].

The standards are assigned different responsibilities. RDF and HTTP IRIs provide the data model and Web identity; OWL provides class and property semantics; SHACL provides explicit validation constraints; SPARQL provides query access; and Fuseki provides a storage/query runtime. Silk is used only for an experimental candidate-linking task. This separation is important because a tool boundary

is not a semantic guarantee: loading RDF into Fuseki does not mint IRIs, SHACL does not prove facts, and OWL profile membership does not certify data accuracy.

Table 2: Design requirements and the mechanism used to address them.

Requirement	Design response	Verification boundary
Interoperability	RDF 1.1, OWL, SKOS, PROV-O, JSON-LD, HTTP IRIs	Syntax and profile checks; external vocabulary semantics are not fully imported
Identity stability	Canonical resource namespace and committed URI registry	URI-regression tests; migration is still required when source keys change
Semantic precision	Separate properties for governance, oversight, membership, branch, ownership, and geography	Domain/range inspection and competency queries
Data quality	JSON Schema, raw asserted checks, OWL 2 RL materialization, and SHACL	Release gate; warnings remain visible and are not silently converted to facts
Reproducibility	Cached inputs, fingerprints, pinned runtime versions, CI workflows, and machine-readable reports	Local reconstruction and Linux Fuseki/Silk workflows

3 Data sources and collection

3.1 Source roles

Wikidata contributes identifiers, labels, institutional attributes, organizational links, and person-related records. Vietnamese Wikipedia contributes infobox fields, descriptions, historical text, and links between articles. DBpedia supplies candidate identity links and some fallback values. Selected reference tables describe province mergers, regional groupings, fields of study, majors, and example programs. Geocoding is used as a fallback for location resolution and is not equivalent to an authoritative campus register.

Table 3: Source roles and integration treatment.

Source or input	Information used	Treatment in the release
Wikidata	Identifiers, labels, types, links, and person/organization attributes	Primary identity evidence when a stable identifier is available; provenance is retained
Vietnamese Wikipedia	Infobox values, article descriptions, revisions, and page links	Structured extraction and revision-specific provenance; not treated as an official register
DBpedia and other linked datasets	Candidate external resources and identity links	Interlinking evidence; uncertain matches remain outside accepted identity assertions
Reference tables	Provinces, mergers, regions, fields, majors, and sample programs	Curated integration inputs; programs are illustrative rather than a complete catalogue
Geocoding fallback	Location resolution when structured identifiers are missing	Convenience support only; not evidence of legal campus status

Wikipedia and Wikidata are interconnected community resources. Agreement between them is useful corroborating evidence, but they must not be described as statistically independent sources. Imported statements can share an origin, and one resource may repeat an error from another. A disagreement log exposes selected conflicts; it does not establish which source is correct.

The collection strategy combines article-category traversal, infobox parsing, and Wikidata queries. Results are retained in JSON snapshots. The repository also ships a compressed HTTP cache. In the

revised default pipeline, a cache miss raises an error instead of silently contacting a changing remote source. A fresh collection is an explicit operation. Dependency installation may still require Internet access; the offline claim applies to cached data collection and reconstruction after dependencies are installed.

3.2 Scope and selection effects

The snapshot is a discovered set of educational organizations, not a census. Filtering removes unrelated articles and some records lacking sufficient cross-source evidence. That policy can exclude legitimate but poorly documented institutions. Institutions with more detailed encyclopedia coverage are more likely to have richer records. The same bias is particularly strong for people linked through education records.

The retained higher-education classification includes name-based heuristics. A title beginning with a familiar institution type is evidence used by the implementation; it is not a substitute for a current legal decision. Counts of higher-education institutions must therefore be read as counts under the project's classification policy.

Academic programs are illustrative hand-entered data. They demonstrate program-to-major queries but do not support claims about a university's complete or current admissions offerings. Similarly, a Wikidata education relation establishes recorded attendance or education, not necessarily completion of a degree. The existing *Alumnus* class must be interpreted with that limitation.

4 Architecture and data management

The implementation follows a layered architecture. Raw source material is kept separate from normalized records; RDF generation is separate from reasoning; and validated output is separate from publication. This organization is similar to the Bronze–Silver–Gold pattern used in other Linked Data projects, but the names describe project artifacts rather than a universal standard. The flow in Figure 1 is the release-level view; the detailed transformation diagram appears in Section 6.

4.1 Layers and contracts

Bronze stores responses and extraction snapshots. Silver contains institutions, people, governing bodies, and provinces in a normalized JSON representation. Gold contains asserted RDF, external links, selected instance-level entailments, and dataset metadata. The layers improve inspectability, but the implementation rewrites snapshot files during reconstruction. Bronze is therefore not an immutable append-only archive; Git revisions provide versioned snapshots.

Silver validation checks schema constraints and selected cross-field rules. The audit adds checks for future years, impossible calendar dates, missing cross-collection references, and invalid leadership references. These checks complement, rather than replace, semantic review of the extraction policy.

After RDF generation, literal ranges and functional-field cardinalities are validated on the raw asserted graph, independently of equality in the closure. The rule engine's own error messages are retained in the consistency diagnostics. SHACL then checks the asserted and retained inferred graph. A failure stops the release step. On success, a validation record binds the checked release to its ontology, shapes, Silver files, component RDF files, and serving graph. The publisher rejects missing or stale validation records.

Fingerprints normalize text line endings to LF so that Git checkout on Windows and Linux does not create false stale-release failures. This is a content-integrity check, not a cryptographic signature or a guarantee against someone deliberately editing both the inputs and validation record. Reproduction

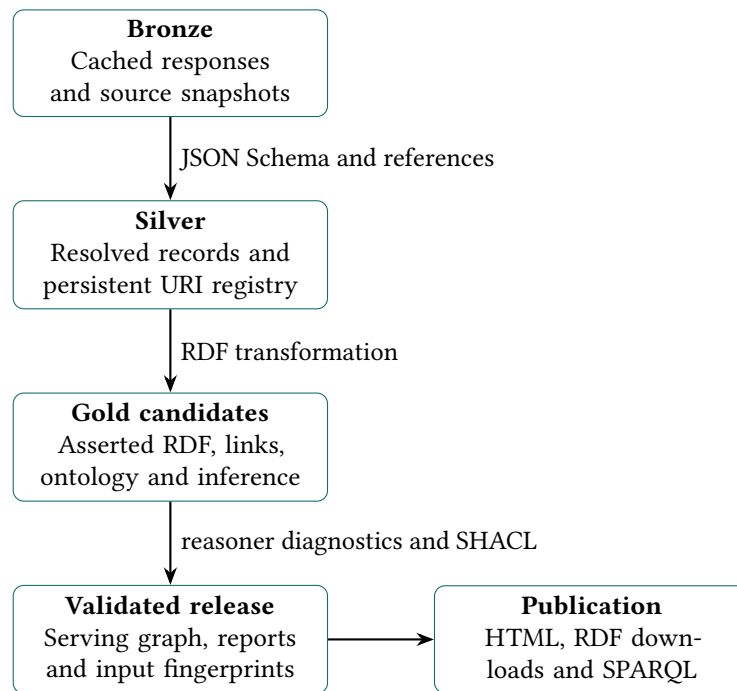


Figure 1: Release flow. A failed validation prevents replacement of validated inference and the serving graph; changed inputs invalidate the publication fingerprint.

of RDF is evaluated semantically; timestamps, blank-node labels, and serialization order can change without changing graph meaning.

4.2 Identity and URI persistence

A readable slug is used when a resource is first assigned a local URI. The revised minter consults the committed `uri_map.json` before allocating anything new. Renaming an entity or inserting another entity with the same label does not change a registered URI. Retired identifiers remain reserved. Source-key changes, such as a new QID replacing an article-based key, still require an explicit identity migration policy.

The previous person integration merged unidentified leaders by normalized personal name across all institutions. The audit found six names used across multiple institutions; splitting these records adds seven person records. This does not prove that seven new real people exist. It represents uncertainty conservatively: records without a shared identifier are separated by institution until identity can be established. The former ambiguous URI is reserved but is not asserted as identical to either replacement.

4.3 Provenance and disputed values

Institution records use `prov:wasDerivedFrom` to reference Wikidata items and specific Wikipedia revisions. Conflict and fallback logs provide additional evidence for some chosen fields. This is entity-level provenance plus partial field-level logging; the project does not yet attach complete provenance to every triple. A Wikidata entity URI is also not a pinned statement revision.

The earliest reported foundation year is the current integration policy and may refer to a predecessor. Five records contain a different year in the infobox foundation date. The revised exporter retains such a date under `vnedu:reportedFoundingDate`, rather than asserting it as an unqualified `schema:foundingDate`. This preserves evidence without pretending that the historical ambiguity has

been adjudicated. Distinguishing predecessor origin, legal establishment, and opening would require dated event records and authoritative supporting sources.

5 System and software architecture

The project is implemented as a batch-oriented Python application with separate data, semantic, publication, and verification responsibilities. Figure 2 shows the complete system from external sources to user-facing artifacts. Figure 3 maps that flow to the repository’s executable components. The diagrams are complementary: the first explains the data life cycle, while the second explains the software boundaries.

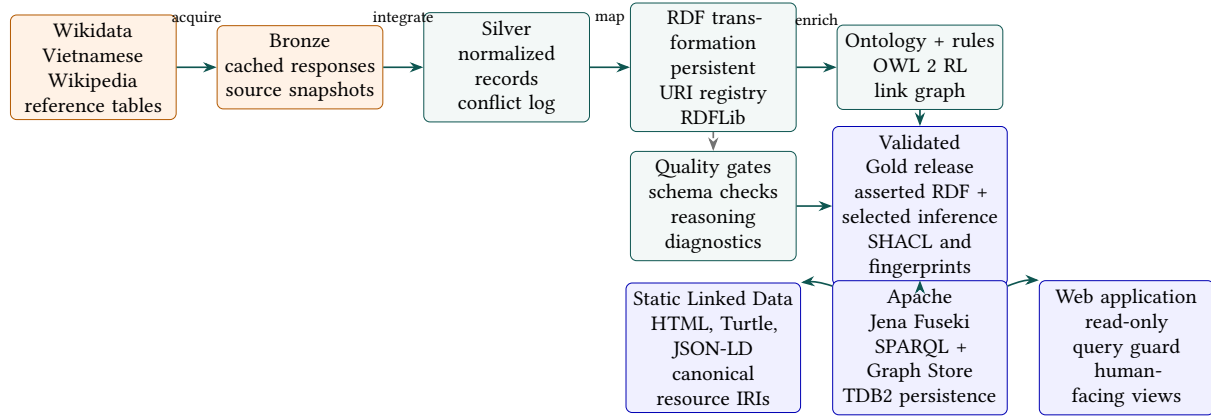


Figure 2: Overall architecture of VN-Edu LOD. Source material is collected and normalized before RDF generation; ontology enrichment and quality gates produce a validated release that can be published statically, queried through the application, or loaded into Apache Jena Fuseki.

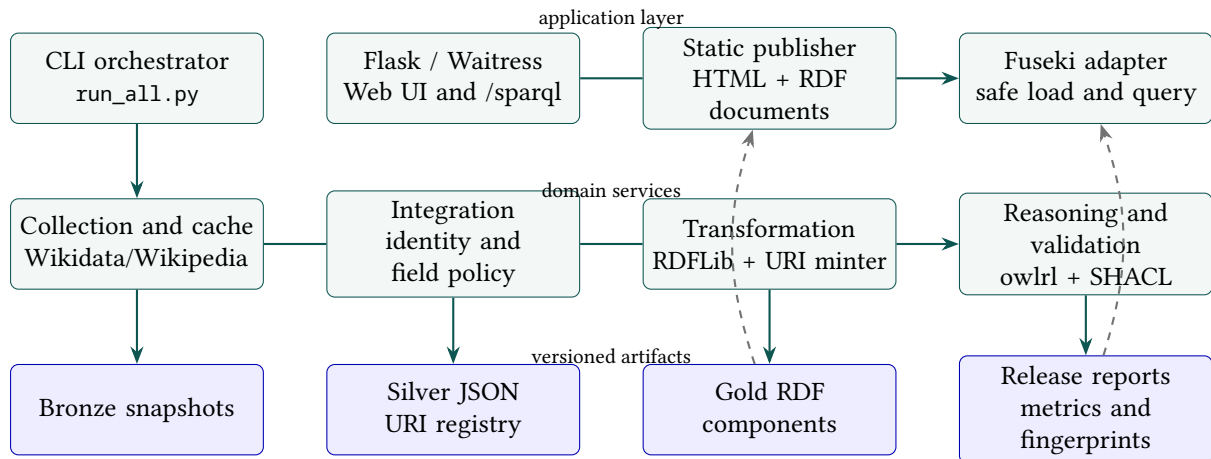


Figure 3: Software architecture. The CLI orchestrator coordinates domain services, while versioned artifacts connect the services to the static publisher, web application, and Fuseki adapter. Dashed arrows indicate artifact hand-off rather than direct runtime calls.

5.1 Component responsibilities

The collection layer obtains structured records and cached source responses. The integration layer applies identity, field-precedence, normalization, and conflict policies. The RDF transformer assigns or reuses resource IRIs, emits typed literals and object links, and keeps the asserted graph separate from

inferred output. The semantic layer applies the local ontology, selected OWL/RDF rules, external links, and qualified observations. The release layer runs raw-data checks, OWL diagnostics, SHACL, metric generation, graph-isomorphism checks, and fingerprint validation before a serving artifact is accepted.

The web application is a consumer of the release rather than the source of truth. Its local RDFLib backend is useful for an isolated read-only view; the optional Fuseki adapter provides the tested HTTP/TDB2 path. The static publisher produces resource pages and RDF downloads for Linked Data clients. This separation makes it possible to test the transformation and semantic artifacts without requiring a running web server.

5.2 Technology choices

Python and RDFLib were retained for transparent transformation and easy regression testing. OWL API and Protégé are used to inspect the ontology and check its OWL 2 RL profile. GraphDB is useful for interactive repository exploration and visualization, but its layout is not the formal specification. Apache Jena Fuseki 6.2.0 is used as the RDF HTTP server and TDB2 persistence target in the end-to-end acceptance workflow. Silk Framework 3.6.0 is isolated as a candidate-linking experiment so that uncertain matches cannot silently enter the release.

5.3 Tooling stack

Figure 4 summarizes the principal tools by responsibility. The short colored marks are typographic identifiers for the tools, included to make the architecture readable in print; they are not claims that the project redistributes vendor logos. The separation is important for interpretation: Protégé and OWL API inspect semantics, GraphDB supports interactive exploration, Fuseki provides the tested HTTP/TDB2 runtime, and Silk produces candidate links that still require review.

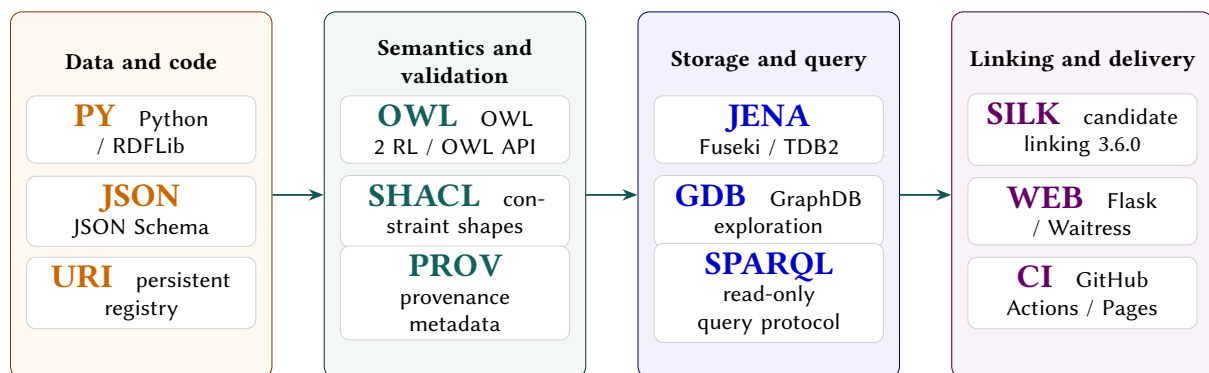


Figure 4: VN-Edu technology stack grouped by responsibility. The left-to-right arrows summarize the implementation path; they do not imply that all tools are runtime dependencies of every request.

Table 4: Main software components and their boundaries.

Component	Responsibility	Boundary
run_all.py and pipeline scripts	Orchestrate collection, integration, transformation, reasoning and publication	Batch execution; not a long-running RDF server
RDFLib and Python validators	Create RDF, apply local policies, inspect graphs, and run regression checks	Does not replace formal OWL profile checking or production load testing
Protégé/OWL API	Inspect ontology axioms and profile membership	Does not decide whether source facts are true
Flask/Waitress application	Read-only UI and guarded SPARQL access	Uses bounded local execution; external TLS/proxy policy remains operational
Apache Jena Fuseki/TDB2	Persistent RDF store and SPARQL/Graph Store protocol	Requires deployment hardening, authentication, backups, and measured capacity

6 Data transformation and knowledge-graph population

The transformation is implemented in Python. Apache Jena Fuseki is not the transformer: it is the RDF store and query server used after the graph has been created. Figure 5 shows the implemented path from source records to the validated serving union.

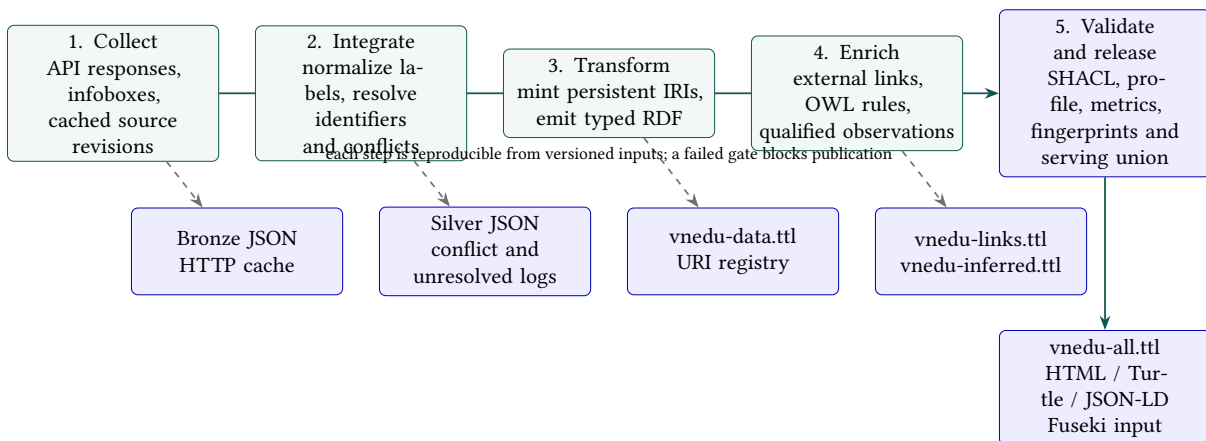


Figure 5: Data transformation workflow. Each stage produces a reviewable artifact. The final serving union is emitted only after RDF, ontology, reasoning, SHACL, metric, and fingerprint checks succeed.

6.1 Bronze layer: collection and source preservation

The Bronze layer stores cached responses, structured snapshots, and source context. Wikidata queries contribute identifiers and structured attributes; Vietnamese Wikipedia contributes infobox fields, article text, links, and revision identifiers; reference tables contribute administrative crosswalks, fields, majors, and illustrative programs. A cache miss fails by default during reconstruction, so a normal build does not silently change its input by contacting a live endpoint. Fresh collection is an explicit operation.

6.2 Silver layer: integration and normalization

The integration stage joins records through stable identifiers when available, normalizes labels and values, resolves organization references, and records conflicts or unresolved targets. It produces normalized

JSON for institutions, people, governing bodies, provinces, and programs. The Silver layer is where source policies are applied; it is not yet an OWL graph. For example, a ministry associated with a private institution is represented as reported state oversight rather than automatically asserted as direct ownership or governance.

6.3 Gold layer: RDF transformation, linking and reasoning

The RDF transformer maps each normalized record to a class and a set of properties in ontology/vnedu.ttl. Persistent IRIs are reused from uri_map.json; a readable slug is only the initial allocation key. Literal values receive explicit language tags and datatypes, while resource-valued fields become object IRIs. External links are written to a separate link graph. The reasoner applies the local ontology and selected OWL/RDF rules, and the release retains selected instance-level entailments rather than treating every internal closure statement as an authoritative source fact.

6.4 Concrete Silver-to-RDF mapping

Table 5 follows the shipped Silver record Q3075696, Hanoi University of Science and Technology. It demonstrates the mapping rules; it is not an independent verification of the institution’s current legal status or leadership.

Table 5: Selected Silver-to-RDF mappings for one institution record.

Silver input	RDF output	Rule or qualification
key = Q3075696	Subject u:dai-hoc-bach-khoa-ha-noi	Reuse the persistent registry; a label change does not mint a new IRI.
kind = University	rdf:type vnedu:University	Classification follows the integration policy, not Fuseki.
name_vi	rdfs:label and skos:prefLabel with @vi	Preserve language information.
founding_year = 1956	vnedu:foundingYear ``1956'' ^^xsd:integer	May refer to predecessor origin; the event meaning remains qualified.
province = Q1858	vnedu:locatedIn the registered Hanoi IRI	Use an object resource, not a province-name literal.
ownership = public	vnedu:ownership vnedu:PublicOwnership	Supports the public-institution classification rule.
viwiki_revid	prov:wasDerivedFrom a Wikipedia oldid IRI	Entity-level source context, not complete assertion-level provenance.

Missing optional values normally produce no triple; they are not converted into empty strings, zeros, or claims of non-existence. Reference programs are identified using the institution slug and major code. This gives a stable demonstration resource, but it does not distinguish every curriculum, degree level, delivery mode, or academic year without additional identifiers.

6.5 Release boundary

The serving graph is the set union of ontology, asserted data, selected inference, external links, and metadata. Component files remain available for inspection, but loading the union into a default graph does not recreate named-graph provenance. The workflow is therefore a reproducible staged batch process, not a transaction covering every intermediate file. The validated release record binds the accepted output to its inputs and rejects stale publication or Fuseki loading attempts.

7 Ontology design and relationship semantics

7.1 Design method

The ontology was designed from competency questions rather than from the visual shape of the source tables. Candidate concepts were grouped into modules, each relation was assigned a domain and range only when the data semantics justified it, and stronger axioms were kept separate from data-quality rules. The resulting model is therefore a TBox: it describes classes, properties, signatures, hierarchies, alignments, and selected entailment patterns. The institutional records, people, programs, and locations are the ABox. SHACL shapes and raw-record checks validate the ABox without pretending that every optional field is complete.

This distinction also explains why the ontology diagram should not be read as a relational database schema. A relational diagram emphasizes tables and cardinalities; an OWL diagram emphasizes concepts, property semantics, and possible entailments under the open-world assumption. The project uses an ER/UML-like projection for readability, while the Turtle axioms, Protégé inspection, profile check, and reasoner output remain authoritative.

7.2 Modules

The ontology has 39 named local classes, 38 local object properties, and 27 local datatype properties in the audited release. Table 6 summarizes the main relationship choices. Figures 6 and 7 visualize the implemented axioms, while Figure 9 shows a populated individual-level neighborhood. Classes, property signatures, and instances are distinguished explicitly.

Table 6: Main modules and intended distinctions.

Module	Main entities or properties	Interpretation
Organizations	Institution, governing body, company; memberOf, branchOf, governedBy, ownedBy	Membership, branch status, governance, and ownership are separate relations.
Oversight	stateManagedBy	General state oversight is distinct from direct governance.
People	Person, InstitutionLeader, Alumnus; rector, director, councilChair	Leadership roles differ; education records do not certify graduation.
Geography	Province, FormerProvince, Region; locatedIn, partOf, mergedInto	Geographic crosswalks support aggregation under successor boundaries.
Education	AcademicProgram, Major, FieldOfStudy; offersProgram, ofMajor, inField	Example programs connect institutions to subjects; these are not an official offering inventory.
Identity	owl:sameAs, skos:closeMatch, foaf:isPrimaryTopicOf	Identity, approximate concept mapping, and document-topic links are not interchangeable.

Disjointness axioms separate selected organizational and geographic categories. Their usefulness depends on defensible category definitions. A statement that violates a disjointness axiom could indicate erroneous data or an overly rigid ontology, so a detected conflict should trigger inspection rather than automatic deletion of evidence.

Vocabulary alignments use subclass and subproperty relationships. Such alignments still carry logical implications; choosing a weaker alignment than equivalence does not make it semantically harmless. The current inference run does not import and validate the complete external ontologies. Its absence of detected conflicts therefore applies to the local reasoning scope, not every possible external combination.

7.3 Ontology overview

The ontology is the conceptual backbone of the graph. It is organized around educational organizations, people, academic programs, administrative units, ownership, governance, and source-qualified observations. Figure 6 is the main readable overview: every local class appears once, while color and edge style distinguish conceptual modules, property relations, and subclass axioms.

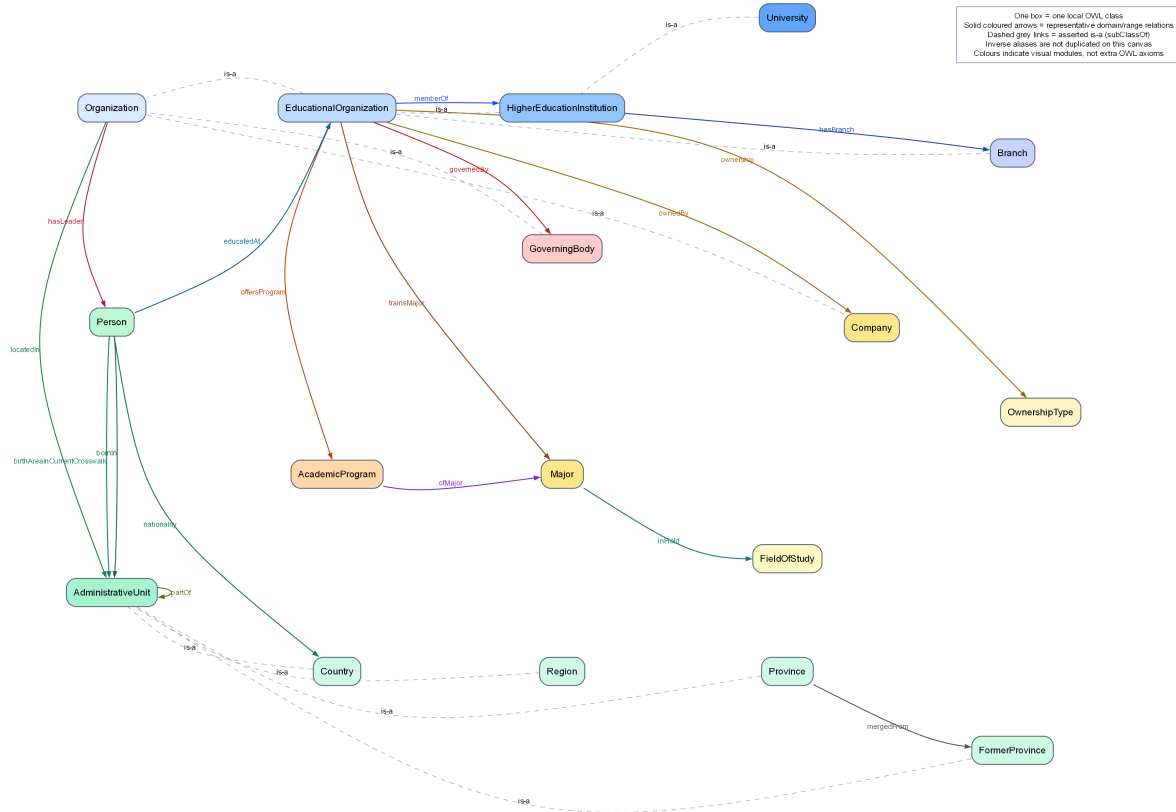


Figure 6: Overview of the VN-Edu ontology. Each local class is shown once; solid edges summarize asserted domain–range relations and dashed edges show asserted `rdfs:subClassOf` axioms. Inverse pairs are represented once for readability. The figure is a deterministic projection of the Turtle source, not a hand-drawn replacement for the OWL model.

The map is intentionally similar to an ER/UML or GraphRAG schema so that a project reader can see the domain at a glance. It is not a relational schema: OWL uses an open-world assumption, property assertions can entail types through domain and range, and omitted triples mean unknown rather than false. The formal model is the Turtle ontology; the diagram is a communication view.

7.4 TBox structure and property semantics

The TBox contains 39 local classes, 38 object properties, and 27 datatype properties. Figure 7 combines the class hierarchy with representative property signatures and one property-chain inference. It is the formal design view: the upper part shows predicates and their domain/range links, while the lower part shows the class hierarchy. Disconnected root concepts are retained because adding a superclass merely to make a picture connected would change the ontology semantics.

The main design decisions are:

- governance, state oversight, membership, branch structure, and ownership are distinct relations rather than one generic parent relation;

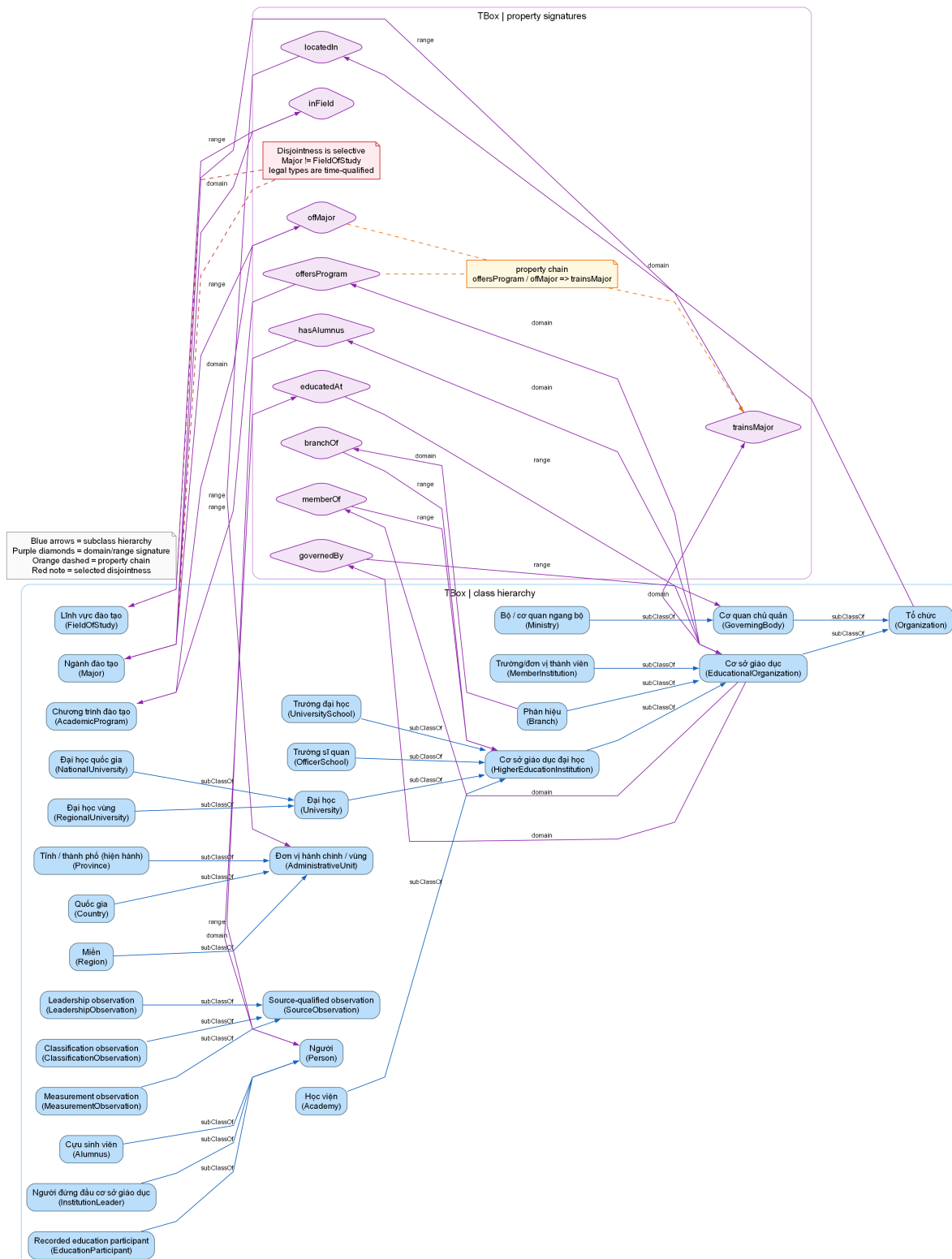


Figure 7: TBox class hierarchy and representative property signatures generated from the committed ontology. Blue edges are subclass axioms, purple diamonds are properties with domain/range links, and the highlighted chain shows the selected offersProgram/offMajor entailment.

- people, leaders, alumni, and education participants are separated so that recorded education does not entail graduation or employment;
- former provinces, current provinces, containment, and merger relations are directed and time-qualified in interpretation; and
- qualified observations preserve source context and optional dates without asserting that an undated leadership or measurement is current.

Table 7: TBox, ABox and validation responsibilities.

Layer	What is asserted	Correct interpretation
TBox	Classes, subclasses, domain/range, inverses, chains, restrictions, alignments and selected disjointness	Schema-level meaning and possible entailments
ABox	Individuals, types, object links, literals, external identifiers and provenance observations	Source-backed snapshot statements; omission is not negation
OWL reasoning	Domain/range and property rules materialize selected types, inverses and chains	Entailment is not a database rejection rule or truth certificate
SHACL	Datatypes, cardinalities, patterns, intervals and observation structure	Closed-world validation against declared shapes

7.5 Protégé inspection and ABox population

The committed Turtle ontology was loaded into Protégé 5.6.7 and inspected with OntoGraf. The screenshot in Figure 8 is direct evidence of that tool inspection; the generated overview above is retained because it shows the complete model more clearly than a workbench viewport. GraphDB is used for interactive repository exploration, while Fuseki is used for storage and end-to-end query acceptance. These tools validate different layers and should not be conflated.

The ABox is illustrated separately from the TBox. Figure 9 uses one real university resource and a small neighborhood of programs, majors, fields, locations, and the reported governing body. This separation prevents the common visualization error in which class names and individual names are drawn as if they were the same kind of node.

7.6 Reasoning and conformance

The implementation uses the `owlrl` rule engine for OWL/RDF rule materialization and an OWL API `OWL2RLProfile` gate for the committed ontology. The gate reports zero profile violations. SHACL reports zero violations, 34 warnings, and one informational result under the current policy. These results establish local conformance to the implemented profile and shapes; they do not establish completeness or factual accuracy of the source records.

Representative entailments include inverse relations, organization-to-province containment, and the chain

$$\text{offersProgram} \circ \text{ofMajor} \sqsubseteq \text{trainsMajor}.$$

The model deliberately avoids using domain/range as if they were SQL foreign-key rejection rules. Operational requirements such as required fields, valid year ranges, and observation interval ordering are enforced by raw checks and SHACL.

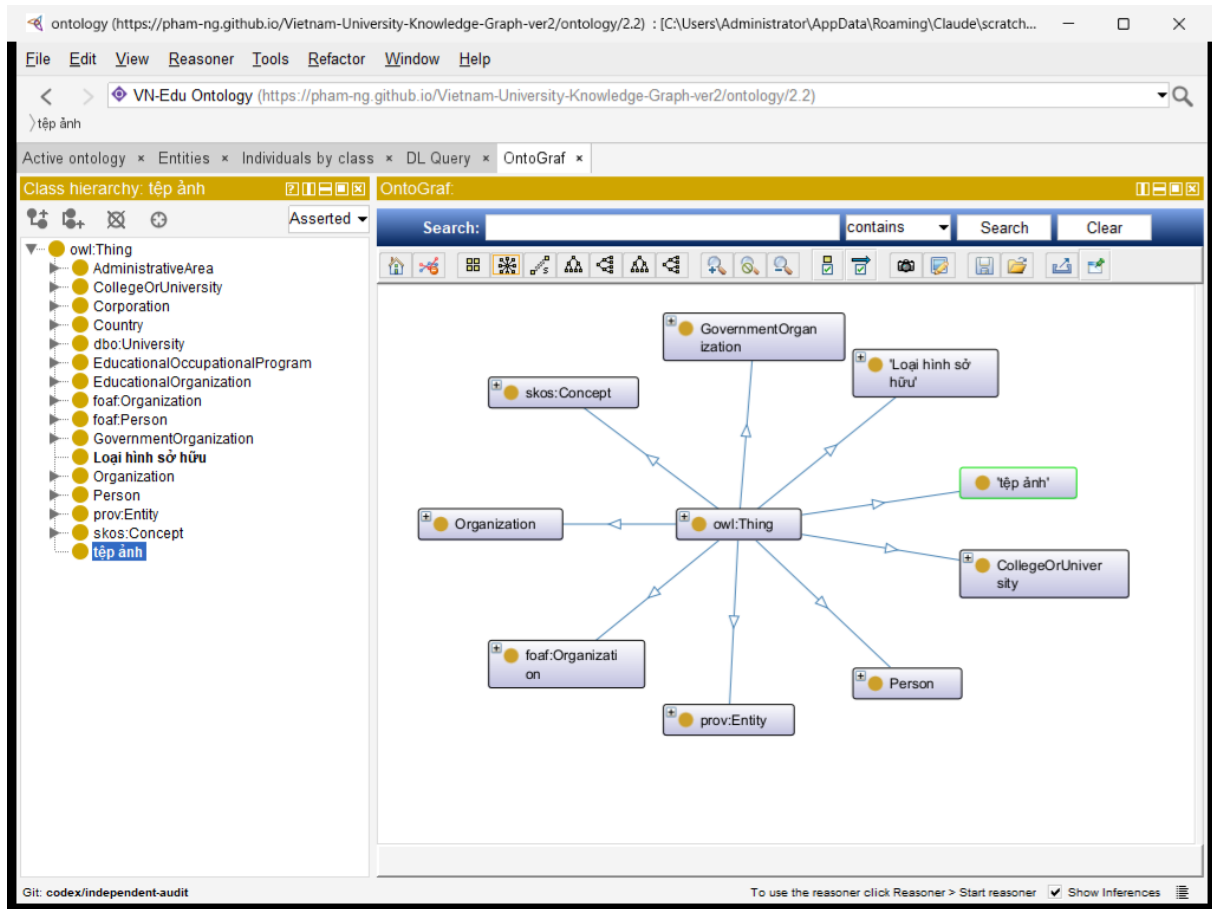


Figure 8: Protégé 5.6.7 OntoGraf view after loading ontology/vnedu.ttl. This is a tool screenshot for verification, not the formal ontology definition.

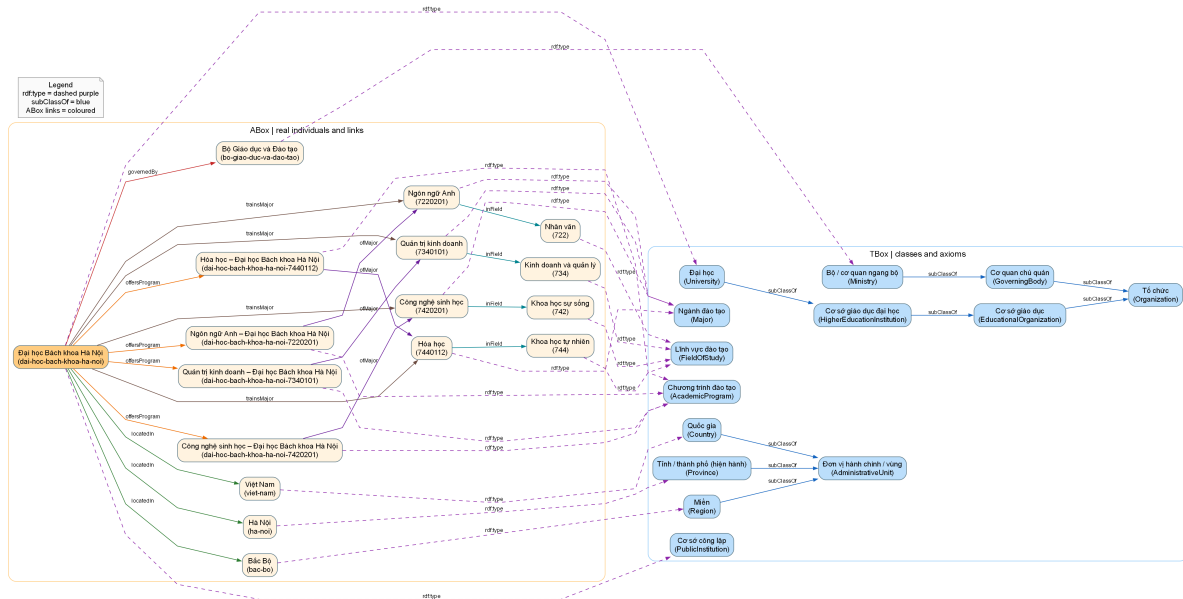


Figure 9: ABox neighborhood for a real university. Blue nodes are classes, orange nodes are named individuals, and the colored links are asserted or retained inferred relations. The view demonstrates population of the ontology without pretending to show the complete graph.

7.7 Materialization and temporal interpretation

The system computes a rule closure over the local ontology and asserted data and retains selected new instance-level statements. External identity links are joined into the serving graph after this step. This avoids equality-driven copying to remote identifiers, but means the published union is not a complete closure including all owl:sameAs effects. The reported inferred-triple count describes the retained subset, not the full internal closure.

Representative rules are:

$$\begin{aligned} locatedIn \circ partOf &\sqsubseteq locatedIn, \\ locatedIn \circ mergedInto &\sqsubseteq locatedIn, \\ governedBy \circ subordinateTo &\sqsubseteq governedBy, \\ offersProgram \circ ofMajor &\sqsubseteq trainsMajor. \end{aligned}$$

For example, an institution assigned to a former province can also be related to the successor province and its larger region. Inverse properties expose the same relationship in the opposite direction without manually storing both directions in the asserted graph.

The model uses the owlrl implementation of OWL 2 RL/RDF rules. A separate OWL API OWL2RLProfile gate checks the committed Turtle ontology; release 2.2 produced zero violations. Calendar years are represented as xsd:integer, and calendar dates as xsd:dateTime at midnight when the source supplies date precision only. This certifies profile membership, not factual correctness, completeness over imported vocabularies, or production performance.

The main temporal limitations are: The birthAreaInCurrentCrosswalk chains are geographic containment crosswalks, separated from bornIn in version 2.2. Relating a birthplace to a province created in 2025 does not mean that province existed when the person was born. Similarly, historical institutions should not automatically be counted as currently operating just because their location can be mapped to a current province. Queries for current institutions should explicitly consider dissolution evidence and its incompleteness.

Version 2.2 adds qualified observations for roles, measurements and classifications, but the available source snapshot still lacks sufficient temporal qualifiers and historical geographic versions. Absence of an end date is not proof of a current appointment. Birth-date collection currently loses source precision, and suppressing January 1 dates is only a heuristic: it can suppress real birthdays and cannot establish the precision of all other dates. These source-quality limitations remain documented work, not conclusions resolved by successful reasoning.

8 Linking and publication

This stage turns the validated graph into a Linked Data artifact. It has three distinct responsibilities: interlink local resources with external datasets, expose useful Web representations, and provide a query/storage runtime. The responsibilities are deliberately not collapsed into one product: the Python transformer creates RDF, the publisher creates resource documents, Protégé inspects ontology axioms, GraphDB is useful for interactive exploration, and Fuseki provides the tested Apache Jena SPARQL/TDB2 path.

8.1 External relations

The link graph contains 3,558 triples. Wikidata, ROR, GeoNames, and selected DBpedia links use owl:sameAs; subject mappings use skos:closeMatch; Wikipedia article links use foaf:isPrimaryTopicOf. The last of these relates an entity to a document and must not be

Table 8: Technology roles in the VN-Edu publication architecture.

Component	Main role	What it does not prove
Python/RDFLib pipeline	Integrates records, assigns URIs, emits RDF, materializes selected entailments	Does not provide a persistent public endpoint by itself
Protégé and OWL API	Inspects ontology axioms and checks OWL 2 RL profile membership	Does not validate all ABox facts or measure production capacity
GraphDB Workbench	Interactive repository loading, visualization, and SPARQL exploration	A layout is not a formal ontology specification
Apache Jena Fuseki/TDB2	HTTP SPARQL/Graph Store access and persistent acceptance path	Does not mint canonical IRIs or establish factual accuracy
Silk Framework	Candidate entity-linking experiment on a frozen sample	Candidate links are not automatically accepted as owl:sameAs

counted as an identity assertion. Repeated attempts to add the same link no longer inflate linkset counters.

Shared identifiers are useful identity evidence, but errors and historical scope mismatches can occur in the source mappings. Some organizational links are discovered through label matching with contextual filters. These deserve separate manual review. The audit does not certify every identity link or every administrative boundary match.

8.2 Representations and deployment

The static publisher generates HTML pages, embedded JSON-LD, and RDF files. The dynamic server provides query handling and RDF content negotiation. The canonical namespace currently belongs to GitHub Pages; the dynamic service has another origin. Content negotiation at the dynamic service does not establish that the canonical static URI performs the same negotiation.

The implementation supports practices associated with five-star Linked Open Data. Section 10 assesses each level and reports a small, dated HTTP sample; it does not reuse the earlier report’s live availability measurements. The repository declares data licensing and retains media metadata; this audit is not a legal verification of the license of every upstream asset.

The publisher now checks its output path before recursive replacement. It refuses the project root, external paths, symbolic links, and existing directories without generated-site markers. This prevents a misconfigured output environment variable from deleting source data or the project itself.

8.3 User-facing resource and query views

The publication layer is also evaluated from the perspective of a reader and a developer. Figure 10 shows a generated resource page for a university. It exposes the canonical IRI, labels, source context, external links, and machine-readable affordances without requiring a user to inspect the raw Turtle file. Figure 11 shows the separate SPARQL interface used to submit read-only competency queries and inspect tabular results. These screenshots document the user interface only; endpoint availability, persistence, graph parity, and the read-only policy are established by the automated acceptance tests described in Section 9.

VN-Edu LOD
Tổng quan
Bản đồ
Tra cứu
Ontology
SPARQL
Dataset
Giới thiệu
Demo
Giao diện

Cơ sở giáo dục, Cơ sở giáo dục đại học, Cơ sở tư thực, Trường đại học, Tổ chức

Trường Đại học VinUni

<https://pham-ng.github.io/Vietnam-University-Knowledge-Graph-ver2/resource/university/truong-da-hoc-vinuni>
Turtle · JSON-LD · Truy vấn SPARQL · (suy luận) = 24 giá trị do bộ suy luận OWL 2 RL sinh ra

Giới thiệu chung

Trường Đại học VinUni (tên đầy đủ là VinUniversity) là một trường đại học tư thực tại Hà Nội, Việt Nam, trực thuộc Tập đoàn Vingroup. Trường được Thủ tướng Chính phủ Nguyễn Xuân Phúc phê duyệt chủ trương thành lập vào tháng 3 năm 2018, và chính thức được cấp phép hoạt động vào ngày 17 tháng 12 năm 2019 với tổng mức đầu tư ban đầu 6.500 tỉ đồng. Vingroup đặt mục tiêu đưa VinUni vào nhóm các trường đại học có thứ hạng cao trên thế giới. Khi thành lập, Trường Đại học VinUni hợp tác với hai trường là Đại học Cornell và Đại học Pennsylvania trong các lĩnh vực quản trị, phát triển chương trình đào tạo, tuyển dụng và kiểm định chất lượng. Khóa đầu tiên của VinUni tốt nghiệp vào tháng 6 năm 2024 với 145 sinh viên.

Lịch sử

Tháng 11 năm 2017, Vingroup bắt đầu hợp tác với Trường Kinh doanh Cornell SC Johnson thuộc Đại học Cornell để khảo sát khả năng thành lập một trường đại học tư thực tại Việt Nam. Tháng 4 năm 2018, Vingroup ký kết thỏa thuận hợp tác với Đại học Cornell và Đại học Pennsylvania, trong đó Đại học Pennsylvania tham gia phát triển các chương trình đào tạo y khoa.

Ngày 14 tháng 11 năm 2018, lễ khởi công xây dựng khuôn viên VinUni được tổ chức tại khu đô thị Vinhomes Ocean Park, Gia Lâm, Hà Nội. Ngày 17 tháng 12 năm 2019, Thủ tướng Chính phủ chính thức ký quyết định thành lập Trường Đại học VinUni. Ngày 15 tháng 1 năm 2020, khuôn viên trường được khánh thành với sự tham dự của Phó Thủ tướng Vũ Đức Đam và Bộ trưởng Giáo dục và Đào tạo Phùng Xuân Nhạ.

► [Đọc tiếp \(5 đoạn\)](#)

Văn bản trích từ bài «[Trường Đại học VinUni](#)» trên Wikipedia tiếng Việt (bản sửa đổi 75478634), giấy phép CC BY-SA 4.0. Trong RDF: `dbo:abstract`, `vnedu:history`.

Trường Đại học VinUni

VINUNIVERSITY

Sử dụng hợp lý

TÊN KHÁC	VinUniversity
TÊN TIẾNG ANH	VinUni
LOẠI HÌNH	Trường đại học · Tư thực
THÀNH LẬP	17 tháng 12 năm 2019 (mốc sớm nhất giữa các nguồn: 2018)
TỔ CHỨC MẸ	Tập đoàn Vingroup
HIỆU TRƯỞNG	Tan Yap-Peng
CHỦ TỊCH HỘI ĐỒNG TRƯỞNG	Lê Mai Lan
ĐỐI TÁC	Đại học Cornell Wikidata Đại học Pennsylvania Wikidata Học viện Công nghệ Georgia Wikidata Đại học Quản lý Singapore Wikidata Đại học Công nghệ Nanyang Wikidata
ĐỊA CHỈ	Vinhomes Ocean Park, Đa Tốn, Gia Lâm

Figure 10: Generated Linked Data resource view for a university.

VN-Edu LOD
Tổng quan
Bản đồ
Tra cứu
Ontology
SPARQL
Dataset
Giới thiệu
Demo
Giao diện

SPARQL trong trình duyệt

Toàn bộ dataset (ontology + dữ liệu + liên kết + suy luận) được nạp vào Oxigraph chạy bằng WebAssembly – SPARQL 1.1 đầy đủ, không cần máy chủ. SPARQL endpoint HTTP công khai (SPARQL 1.1 Protocol, hỗ trợ federated): <https://vnedu-lod.onrender.com/sparql>; bản Apache Jena Fuseki chạy cục bộ theo hướng dẫn trong repo.

Truy vấn mẫu

[Suy luận] Số cơ sở GDĐH theo 34 tỉnh/thành sau sắp xếp 2025 – trường ghi ở tỉnh cũ được tự quy về tỉnh mới

[Suy luận] Cơ sở GDĐH theo miền và loại hình sở hữu (miền suy ra qua chuỗi locatedIn ◦ partOf)

[Suy luận] Trường quân đội & công an – phân loại tự động bằng lớp định nghĩa (governedBy ◦ subordinateTo)

Tra cứu theo mã trường tuyển sinh (VD: BKA, KHA, NTH, YHB, QHI)

[Suy luận] Các cơ sở có đơn vị thành viên – hasMember là nghịch đảo được suy ra từ memberOf

Sắp xếp đơn vị hành chính 2025 (NQ 202/2025/QH15): tỉnh mới, các tỉnh cũ hợp nhất vào, số cơ sở ghi ở tỉnh cũ

Hồ sơ đầy đủ của Đại học Bách khoa Hà Nội (mã BKA) – gồm thông tin suy luận và liên kết ra ngoài

[Suy luận] Trường đào tạo ngành Công nghệ thông tin (trainsMajor = offersProgram ◦ ofMajor; chương trình là dữ liệu mẫu)

[Suy luận] Trường có nhiều cựu sinh viên nổi tiếng nhất (lớp Alumnus suy ra từ alumnusOf)

[Suy luận] Người đứng đầu các đại học quốc gia & vùng (rector/director ◦

```
# [Suy luận] Cơ sở GDĐH theo miền và loại hình sở hữu (miền suy ra qua chuỗi locatedIn ◦ partOf)
PREFIX vnedu: <https://pham-ng.github.io/Vietnam-University-Knowledge-Graph-ver2/ontology#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

SELECT ?mien (COUNT(DISTINCT ?u) AS ?tong) (COUNT(DISTINCT ?cl) AS ?cong_lap) (COUNT(DISTINCT ?tt) AS ?tu_thuc)
WHERE {
  ?u a vnedu:HigherEducationInstitution ; vnedu:locatedIn ?m .
  ?m a vnedu:Region ; rdfs:label ?mien FILTER(LANG(?mien) = "vi")
}
```

Chạy (Ctrl+Enter)
☐ chạy trên máy chủ
3 kết quả · 0 ms

MIEN	TONG	CONG_LAP	TU_THUC
Bắc Bộ @vi	136	116	10
Nam Bộ @vi	79	56	17
Trung Bộ @vi	48	37	7

Figure 11: Read-only SPARQL query view and returned result table.

9 Deployment and Apache Jena Fuseki

This section documents how the validated RDF release is published and queried. It is useful to separate the URI design from the runtime: a URI identifies a resource, while Fuseki stores and queries triples about that resource. The same graph can therefore be published as static RDF and loaded into Fuseki without changing its identifiers. Fuseki is a SPARQL server, not a URI minting service [9]. The transformer assigns canonical identifiers before data enters any store. The same university IRI remains in RDF whether queried by RDFLib, Oxigraph or Fuseki.

9.1 Identity, representation and storage

```
# RDF subject identifying a university:
https://pham-ng.github.io/Vietnam-University-Knowledge-Graph-ver2/resource/university/dai-hoc-
↳ bach-khoa-ha-noi
# Local query endpoint:
http://localhost:3030/vnedu/sparql
# Graph Store Protocol target:
http://localhost:3030/vnedu/data?default
```

Uploading RDF does not make its canonical IRIs dereferenceable automatically. The publisher independently creates HTML, Turtle and JSON-LD descriptions. Ontology terms retain hash IRIs; entity and observation identifiers use slash paths. Immutable 2.1 and 2.2 version artifacts are now generated under ontology/. Dynamic version routes support RDF content negotiation and return 404 for unknown versions.

GitHub Pages remains a separate HTTP origin from Flask. Its static representation links and embedded JSON-LD do not implement Flask’s negotiation, nor the entity-to-document 303 pattern discussed in *Cool URIs* [6]. Adding Fuseki cannot fix those origin-level behaviors. Version artifacts are implemented and tested locally; their production deployment requires promotion of the review branch.

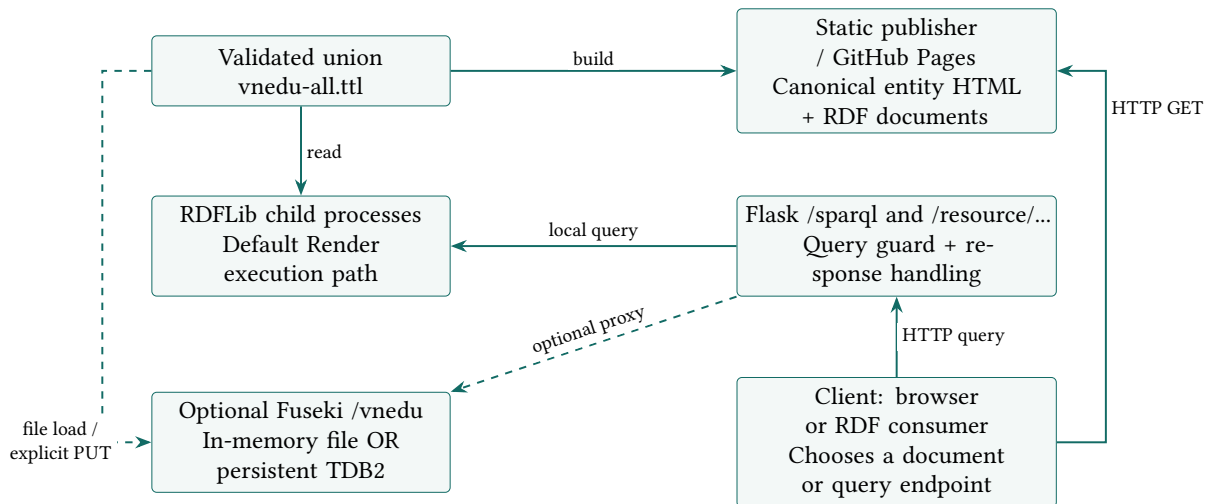


Figure 12: Publication and storage paths. The optional Fuseki branch has verified Linux and Windows acceptance runs; Render still explicitly selects local RDFLib. Storage does not replace the canonical identifiers.

9.2 Safe loading and reproducible runtime

The revised loader requires a successfully validated release before network writes. It targets an existing explicitly named dataset rather than creating one implicitly. An empty graph can be loaded directly.

Replacement of a nonempty default graph requires both `--replace` and a new `--backup` path; the backup is created exclusively, so an older backup cannot be silently overwritten. HTTP redirects and error responses are rejected. After PUT, the loader retrieves the entire default graph and checks RDF graph isomorphism, including blank nodes, rather than relying on a triple count.

Graph Store PUT replaces the selected graph [11]; named graphs are untouched by this loader. Credentials are read from environment variables, not command-line passwords. A failed comparison preserves the backup for operator recovery; it is not a distributed transaction or automatic rollback. Operators must stop concurrent writers during a release: a backup followed by PUT does not provide compare-and-swap concurrency control.

The runtime setup pins Apache Jena Fuseki 6.2.0 and verifies its published SHA-512 digest. The local launcher requires Java 21, validates the release, binds to loopback, and enables only query access with a configured timeout. Its file mode reloads the graph at startup; the separate acceptance harness explicitly tests persistent TDB2. The optional legacy Docker recipe now requires an operator-selected image and password, and binds port 3030 only on loopback. That Docker configuration was not the tested runtime.

9.3 Production-like security and performance acceptance

The additional `audit/production_readiness.py` test starts the same Waitress WSGI entry point used by `app/server.py --prod` with the read-only local backend. Under normal bounded concurrency (eight requests, two query workers), all eight health requests returned HTTP 200; the 8 October 2026 rerun measured 3,965 ms at p50, 3,996 ms at p95, and 3,996 ms maximum on the Windows review machine. An intentional eight-request overload returned two HTTP 200 responses and six HTTP 503 responses, demonstrating bounded back-pressure rather than unbounded queue growth. Security probes rejected a path-injection attempt (404), an unapproved SSRF SERVICE target (400), and an oversized query (413). The response headers now include `nosniff`, `SAMEORIGIN`, `strict referrer-policy`, `Permissions-Policy`, `CSP on HTML`, and an `X-Request-ID`; HSTS is enabled when TLS is detected or explicitly forced behind a trusted TLS terminator.

These numbers are a reproducible loopback acceptance sample, not a capacity SLO, penetration test, TLS/reverse-proxy audit, cloud IAM review, or production security certification. The process-per-query RDFLib backend deliberately trades throughput for isolation; a real deployment still needs external rate limiting, resource quotas, TLS termination, monitoring, authentication policy, and a load test against its selected Fuseki/WSGI topology. The full machine-readable evidence is `data/reports/production-readiness.json`.

9.4 Production hardening profile and release gates

The review branch adds a deployable reference profile in `deploy/` and records its scope in `docs/production-readiness.md`. The intended production path is:

```
Internet -> Caddy/Nginx TLS -> Waitress application -> private Fuseki -> encrypted TDB2 volume
```

The application layer is read-only for SPARQL, limits request bodies and serialized result size, applies an optional per-client rate limit, restricts CORS to an explicit allow-list, and emits request identifiers for incident correlation. The reference Compose file also uses a non-root application user, a read-only container filesystem, a private network, dropped Linux capabilities and a no-new-privileges flag. Fuseki image digests and admin credentials are deliberately operator-supplied; the public reverse proxy must never expose Fuseki administration or update endpoints.

These are implementation controls, not evidence that the stack has been deployed on a public production host. Before a production availability claim, the deployed Fuseki path must be tested for sustained and burst requests, p50/p95/p99 latency, CPU/memory/disk growth, query cancellation, rolling restart and backup restoration. The current local RDFLib p95 is therefore reported as an isolation/back-pressure observation, not as a capacity target.

Table 9: Production release gates that remain separate from OWL and SHACL conformance.

Gate	Required evidence before an authoritative production claim
Security deployment	Public TLS termination, trusted proxy configuration, firewall/IAM policy, secret rotation and external rate limits.
High-load performance	Deployed Fuseki benchmark with representative SELECT/CONSTRUCT queries, p50/p95/p99, saturation behavior and resource measurements.
Recovery	Backup restore, rolling restart and graph-isomorphism verification after replacement.
Factual accuracy	Independent adjudication of the five founding-year conflicts and seven unresolved values, plus a held-out identity-link evaluation.
Temporal validity	Claim-level provenance and validity intervals for leadership, enrolment and legal status; undated snapshots must not be presented as current.
Source authority	Official-register or institution-owner evidence for records currently supported only by encyclopedic or inferred sources.

The release can pass the OWL 2 RL profile gate, SHACL validation and regression tests while still failing these operational and factual gates. Keeping those gates separate is essential: OWL entailment is not a truth certificate, and a five-star Linked Data publication is not a five-star security or accuracy certification.

9.5 Executed end-to-end acceptance evidence

The primary reproducible test ran on Linux GitHub Actions with Fuseki 6.2.0 and Temurin Java 21.0.12.1, on 7 October 2026 at 01:50 UTC. A second run on the Windows desktop used the same Fuseki JAR, Java 21 and the Windows selector/socket workaround at 07:22 UTC; the launcher smoke test passed at 07:31 UTC. The tested code revision before these launcher edits was `ab0033d`; the current branch records the subsequent launcher fix. The input hash, JAR hash, runtime details and assertions are archived in `data/reports/fuseki-e2e-linux.json` and the Windows run in `data/reports/fuseki-e2e-windows.json`; the corresponding Linux CI run completed successfully.

The test uses a newly created disposable directory and ephemeral loopback port; no existing user dataset is overwritten. Zero dated-leadership rows is an honest reflection of missing date evidence, not a failed lookup to be filled with invented dates.

On Windows, the first attempt exposed a JDK 21 selector/socket initialization failure. The repaired launcher now sets the selector provider and short Unix-domain socket directory through inherited Java options (avoiding PowerShell 5.1 argument concatenation); the same disposable TDB2 acceptance test

Table 10: Measured Fuseki/TDB2 acceptance results.

Check	Observed result
Load complete validated serving graph	73,628 triples; graph-isomorphic to local RDF.
Attempt unapproved replacement	Refused before PUT.
Approved replacement with backup	Backup graph-isomorphic to previous contents.
Query parity with RDFLib	Count: 1 row; class totals: 68 rows; programs: 78 rows; explicitly dated leadership: 0 rows.
Stop and restart with the same TDB2 directory	Entire default graph remains isomorphic.
Restart without update permission	SPARQL Update and Graph Store PUT both return HTTP 405.
Read after rejected writes	Graph unchanged and isomorphic.

then passed all checks, and a direct query-only launcher smoke test returned HTTP 200. This proves local Windows operability, not production deployment. Configured timeout is not proof of cancellation under load. Authentication, SERVICE egress, concurrent writers, memory limits, disaster recovery and production TLS require separate acceptance tests. The verified read-only mode is access restriction, not a full security certification.

10 Five-star Linked Open Data assessment

The five-star scheme evaluates how data is published and connected, not the overall quality of a program or the correctness of every ontology axiom [7]. There is no Fuseki-specific requirement: RDFLib or static RDF publication can support the relevant mechanisms. Table 11 assesses the project’s data publication rather than assigning an unsupported software-quality score.

Table 11: Evidence against the cumulative five-star publication scheme.

Level	Criterion, summarized	Project evidence and qualification
1	Publicly accessible, openly licensed	Public sample accessible; dataset metadata declares CC BY-SA 4.0. Upstream media retain their own licenses.
2	Structured, machine-readable	JSON records and parsed RDF graphs, not only screen images or PDFs.
3	Non-proprietary format	JSON, Turtle and JSON-LD artifacts.
4	Web identifiers and open Semantic Web standards	Stable-registry HTTP IRIs, RDF descriptions and query interfaces; sampled entity documents parse. Canonical negotiation remains static; version artifacts are now implemented on the review branch.
5	Connections to other datasets	Corrected local link graph contains 2,309 owl:sameAs, 61 skos:closeMatch and 1,188 document-topic links. Existence is measured; every identity assertion is not independently verified.

Verdict. The implementation and sampled public RDF support the basic five-star publication pattern, subject to the declared licensing scope. This is not an independent certification, proof of factual accuracy, or proof that the corrected branch is deployed. Deployment, temporal-evidence and operations limits should be reported alongside the claim, not hidden by a five-star badge. Failing a single endpoint probe does not automatically reduce an otherwise accessible RDF dataset to a lower star level.

10.1 Dated HTTP evidence

audit/check_publication.py performs seven read-only requests with bounded timeouts; the result is saved in data/reports/publication-check.json. This historical, pre-migration sample was recorded

on 7 October 2026 at 00:10 ICT (UTC+7). The institution tested was `resource/university/dai-hoc-bach-khoa-ha-noi`; this is not a random or exhaustive sample.

Table 12: Observed HTTP behavior; deployed content may predate the corrected branch.

Request	Observation
Canonical entity, Accept HTML	HTTP 200, HTML, embedded JSON-LD and alternate RDF links.
Canonical entity, Accept Turtle	HTTP 200, still HTML; no HTTP redirect.
Entity's .ttl document	HTTP 200, Turtle, 204 triples; graph describes the entity IRI.
Entity's .jsonld document	HTTP 200, JSON-LD, 204 triples; graph describes the entity IRI.
Declared ontology/2.2 version	Source build emits immutable HTML, Turtle and JSON-LD; live Pages availability awaits deployment of this corrected branch.
Render /healthz	Read timeout after 15 seconds; health and deployed commit were not established.
Local Fuseki /\$/ping	Connection refused on port 3030; no reachable service at the configured address.

A Render timeout can reflect a cold start or a network/service problem; it is not proof of permanent downtime. The probe did not test SPARQL federation, persistence, admin credentials, or the full set of resource identifiers. The existing public deployment remains distinct from the audited branch, whose Linux CI and local tests validate a different scope.

The media distinction also matters. For example, the shipped HUST logo metadata records “fair use” while its campus photograph records CC0. Describing or linking such an image does not relicense the image under the dataset license. The revised publisher shows a source link, not an embedded image, for unknown, fair-use or unrecognized licenses. A data publication claim must not be expanded into a blanket claim that every remotely displayed asset is openly licensed.

11 Application security and query execution

The original SPARQL policy searched for textual patterns containing whitespace after `SERVICE` and `FROM`. Valid forms without that whitespace bypassed the filter. The audit reproduced this locally using loopback targets without issuing an outbound request. The revised policy inspects parsed SPARQL operations and permits only two exact HTTPS service endpoints. It rejects dataset-loading clauses, variable service targets, alternate ports, credentials, and arbitrary paths. Security keywords inside string literals are not treated as operations. The local outbound HTTP handler also rejects redirects.

The previous timeout waited only for `Graph.query` to return. `RDFLib` can evaluate `SELECT` bindings lazily, so result evaluation could continue beyond the limit. Cancelling a future also cannot stop an already running Python thread. The revised local backend evaluates and serializes each query in a disposable process, terminates it on timeout, and limits concurrent workers. An expensive Cartesian-product regression test checks that capacity is released and a subsequent small query succeeds.

The local backend limits serialized responses to 8 MiB. This does not impose a hard memory limit during intermediate evaluation, and parsing the public request still occurs in the web process. A hostile-workload production deployment therefore needs process or container memory limits, request-rate controls, and appropriate upstream isolation. The process-per-query approach also reloads the graph and has a throughput cost. Earlier in-memory latency measurements do not describe this implementation.

The Fuseki proxy supplies an execution timeout parameter and does not follow proxy redirects. Actual query cancellation and remote `SERVICE` egress remain responsibilities of the configured Fuseki instance. The separate Linux acceptance run verifies load/query/persistence/read-only behavior; it does not verify production federation controls. Raw POST requests with `application/sparql-query` are now accepted, alongside form-encoded query requests.

The security tests support a bounded local read-only service: path traversal, unapproved federation targets, oversized queries, and uncontrolled worker growth are rejected by the implemented checks. These are application-level controls. They do not replace TLS termination, authentication, external rate limiting, memory quotas, secret rotation, monitoring, or a deployed Fuseki load test. The production topology and its remaining gates are documented in Section 9.

12 Reproducible workflow and CI/CD

The repository uses continuous integration as a reproducibility mechanism, not as a substitute for scientific interpretation. Figure 13 shows the ordinary release path and the two specialized evidence paths. The main workflow rebuilds the graph and static site, while Fuseki and Silk run in separate workflows with their own runtime requirements.

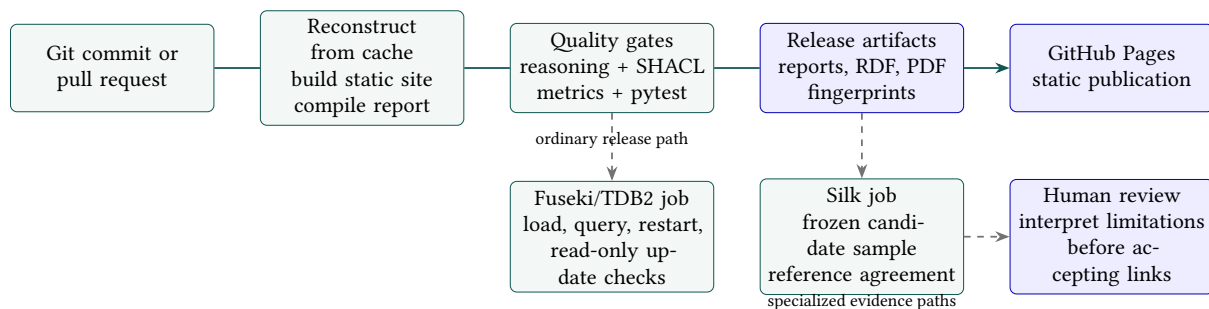


Figure 13: CI/CD workflow. A commit enters the ordinary reconstruction and quality gates; successful artifacts can be published to GitHub Pages. Fuseki/TDB2 and Silk are separate evidence paths with different acceptance claims.

12.1 Release sequence

The release sequence is intentionally ordered. Inputs are reconstructed first, RDF is then materialized and checked, and only a release with matching fingerprints can be published or loaded into Fuseki. A failure in a later gate does not turn a candidate graph into a public graph. This design addresses a common failure mode in data projects: reporting a successful transformation before validating the exact artifact that is served.

1. Reconstruct cached inputs and validate Silver records.
2. Generate asserted RDF, link metadata, ontology data, and selected inferred triples.
3. Run OWL diagnostics, SHACL, metrics, stale-release checks, and graph-isomorphism checks.
4. Build the static publication and the report from the accepted artifact.
5. Run the Fuseki/TDB2 acceptance job and, when requested, the isolated Silk experiment.

12.2 What CI proves

The current suite demonstrates reproducible execution of the declared local gates. It does not prove that every institution is represented, every identity link is correct, or that the public endpoint can sustain an arbitrary workload. Those claims require independent annotation, authoritative source comparison, and a load test against the selected production topology.

13 Evaluation and results

13.1 Release measurements

All values in Table 13 are generated by `audit/release_metrics.py` from the validated local release. That script also checks that the serving graph is isomorphic to the union of its declared components. It does not request data from a remote endpoint.

Table 13: Measured size of the corrected local release.

Quantity	Count
Educational-organization records	300
Classified higher-education institutions	271
Person records	1,702
Institution-leader records	221
People with recorded education relations	1,481
Current / former province-level units	34 / 29
Illustrative programs / majors	78 / 37
Asserted data triples	47,291
Retained inferred triples	21,825
Link-graph triples	3,558
Ontology / dataset-metadata triples	862 / 96
Distinct triples in serving union	73,628
Local resource IRIs appearing as subjects	5,401

In the corrected snapshot, selected SHACL checks report 0 violations, 34 warnings, and 1 informational result. Warnings include missing foundation years and institution locations. Warning-tolerant conformance means the configured release policy permits these omissions; it does not mean all records are complete. The consistency diagnostics report no detected local violations after the fixes.

Field coverage uses all 300 institution records as its denominator: 278 have a selected foundation year, 288 a province reference, 256 an ownership value, 237 a website, 175 coordinates, 60 an admission code, and 28 a student-count value. These are presence measurements, not estimates of truth, timeliness, or national coverage. In particular, 271 higher-education classifications and 300 educational-organization records are different denominators and should not be interchanged.

13.2 Link experiment: agreement, not independent accuracy

The available experiment uses 86 institutions with English labels and identifier-derived DBpedia reference targets. A local token-Jaccard matcher selects the highest-scoring target from that closed reference candidate set. Table 14 reports agreement with the reference frozen from revision 58c65f0. This baseline is not Silk; the separate actual Silk execution is reported below.

Table 14: Token-Jaccard agreement with the identifier-derived reference.

Threshold	Predictions	Disagreements	Ref. precision	Ref. recall	Ref. F1
0.2	85	13	84.7%	83.7%	0.842
0.4	80	8	90.0%	83.7%	0.867
0.6	76	6	92.1%	81.4%	0.864
0.8	68	4	94.1%	74.4%	0.831
1.0	58	1	98.3%	66.3%	0.792

Let M be the set of predictions, R the reference pairs, and $TP = |M \cap R|$. The reported precision is $TP/|M|$, recall is $TP/|R|$, and F1 is their harmonic mean. Calling R independent ground truth would be incorrect because the identifier linker generated it. Scoring that same linker against R yields perfect agreement by construction. The previous claim of 100% linking accuracy is therefore withdrawn.

The selected sample excludes unlinked institutions and targets outside the reference set. These restrictions affect task difficulty and coverage. The threshold with the highest F1 on this sample is a descriptive observation, not a validated threshold chosen on a separate development set. A defensible accuracy study needs independently annotated links and non-links, dated evidence, an adjudication procedure, and a held-out test set.

13.3 Executed Silk Framework experiment

A separate experiment now executes actual Silk Framework 3.6.0, rather than referring to the Python Jaccard baseline as Silk. The official single-machine entry point was compiled with Scala 2.12.10 against released workbench libraries and run on Java 8. Pinned checksums are in `audit/setup_runtimes.ps1`; the runner, XML rules, input N-Triples, candidate outputs and result JSON are included in the repository. The reproducible execution is scheduled in `.github/workflows/silk-e2e.yml` on Ubuntu with Java 8. On the reviewed Windows desktop, Java 8.351 and 8.481 both timed out before producing candidates; this is recorded as an environment limitation, not converted into a success claim.

The experiment freezes the 86-source/86-target sample from revision 58c65f0, giving 7,396 possible pairs. Both methods use the same lowercased English labels. Silk uses edit-distance thresholds of 0, 3 and 8 with a top-one filter, while the Python baseline uses token-Jaccard similarity. Their threshold values are not numerically comparable. The XML filter is inside `LinkageRule`, as required by the 3.6.0 parser; assertions verify at most one prediction per source and the experiment-only predicate. Older example configurations are not assumed equivalent.

Table 15: Actual Silk versus the fixed-sample Python baseline: reference agreement only.

Rule	Predicted	Matches	Ref. precision	Ref. recall	Ref. F1
Silk: distance 0	57	57	100.0%	66.3%	0.797
Silk: distance 3	64	63	98.4%	73.3%	0.840
Silk: distance 8	75	70	93.3%	81.4%	0.870
Jaccard: similarity 0.4	80	72	90.0%	83.7%	0.867
Jaccard: similarity 0.6	76	70	92.1%	81.4%	0.864
Jaccard: similarity 0.8	68	64	94.1%	74.4%	0.831

These results do not estimate independent identity accuracy. Some historical reference identities are rejected by the current stricter linker; they remain in the frozen experimental reference, not in accepted release identities. The sample excludes open-world negatives, lacks independent adjudication and has no held-out tuning set. Silk’s tie handling is engine-defined; Python uses descending target URI for ties. Small differences do not establish general superiority.

No prediction is promoted to `owl:sameAs`. Silk emits a candidate-only URN predicate and its files are outside the serving pipeline. It is a research extension, not a normal publication dependency. Neo4j is not added: a second graph model and synchronization path would supply neither missing time evidence nor another five-star requirement. The implementation follows the official Silk source and link-specification documentation [8].

13.4 Regression evidence

The audit adds counterexamples for compact SPARQL syntax, exact service endpoint policy, keywords inside literals, raw query POSTs, cancellation and worker recovery, datatype diagnostics, literal ranges, publication gates, stale release records, persistent URIs, same-name identity collisions, offline cache misses, impossible dates, and dangerous publisher paths. Eight initial counterexamples failed before correction and passed after the relevant fixes. The final test-run result and command are recorded in the companion audit notes rather than a manually maintained badge.

The full pipeline is reconstructed from the shipped cache, and both static-site variants are built before the server integration suite. A successful run establishes that the tested release follows the intended local contracts. It does not constitute a penetration test, an independent factual annotation study, or a load test of the public deployment.

On the review branch, the final local regression command completed with 139 passed tests and one dependency deprecation warning. The reasoning gate, release metrics, publication build, and the dedicated Fuseki and Silk workflows are reported separately because they exercise different systems. This is a deliberate evidence matrix rather than one composite “green means correct” score.

13.5 Treatment of earlier comparisons

The earlier report’s 21-fact comparison with a predecessor repository is a small, hand-selected demonstration. Its benchmark script relies on a machine-specific path and permissive answer matching, and the underlying cases do not have a complete independent annotation protocol. Its scores are not used here as an accuracy estimate. Claims about 349 predecessor contradictions, graph-density superiority, live link availability, and public-endpoint latency are likewise not repeated as newly verified findings.

Graph density is especially sensitive to which nodes and predicates are counted. Adding external nodes or inferred inverse edges changes the measurement without necessarily adding useful or correct information. A future comparison should define a common node universe, edge projection, deduplication policy, and frozen input revisions before reporting degree statistics.

14 Discussion, limitations and future work

The evidence supports the project as a reproducible Semantic Web implementation and a useful educational knowledge graph. It does not support a single overall “five-star software” rating. The five-star Linked Open Data scheme concerns how data is published and connected; OWL 2 RL concerns ontology profile membership; SHACL concerns declared constraints; and production readiness concerns security, operations, recovery, and load. A sound evaluation must report these dimensions separately.

Construct validity. Logical consistency, SHACL conformance, field presence, and link agreement measure different things. None alone measures real-world correctness. Program examples and name-based legal-type classification further restrict what answers can be interpreted as authoritative.

Internal validity. Source precedence can select a plausible but incorrect value. Earliest-year selection conflates events unless the event meaning is modeled explicitly. Some unqualified dates and organizational relationships need source-level adjudication. The revised person split prevents unsupported merges, but may also separate records belonging to the same real person; this is an explicit uncertainty trade-off.

External validity. Coverage favors institutions and people documented in the selected community sources. The 86-link experiment omits open-world candidates and cannot be generalized to all Vietnamese institutions. National completeness requires comparison with a dated authoritative institutional register.

Operational validity. Local tests do not prove public availability, deployed revision identity, traffic capacity, or Fuseki egress behavior. Memory limits and endpoint rate controls remain deployment concerns. Snapshot timestamps record reconstruction or collection events as implemented; they must not automatically be interpreted as the dates on which the underlying facts became true.

The next substantive data work is acquiring real temporal qualifiers for the new observations, event-specific foundation histories, and independently verified assertion-level provenance. A subsequent research evaluation should use a preregistered sampling method, an independently reviewed reference set, and a clear distinction between discovery coverage, linking coverage, and correctness. Those additions require evidence not present in the current repository and are not supplied by inventing dates, labels, or outcomes.

15 Conclusion

VN-Edu LOD demonstrates an end-to-end Linked Data workflow for Vietnamese higher education. The project connects a modular OWL ontology to a layered transformation pipeline, persistent HTTP identifiers, external links, reasoning, SHACL validation, publication, and SPARQL access. The corrected release contains 73,628 distinct serving triples, and the automated gates provide reproducible evidence for its local structure, release integrity, and runtime integration.

The most important result is methodological: the report makes the boundaries between claims explicit. OWL 2 RL profile membership is not factual accuracy; SHACL conformance is not completeness; Silk candidate agreement is not an independently measured identity accuracy; and Fuseki acceptance is not a production security or high-load certificate. With those boundaries preserved, the repository is a credible educational Semantic Web artifact and a sound basis for future work using authoritative registers, claim-level temporal provenance, held-out link annotations, and a deployed Fuseki benchmark.

A Reproducing the release and report

Use Python 3.12 and the pinned dependencies. After installation, the default pipeline uses the shipped cache and fails on missing cached responses. Run from the repository root:

```
python -m pip install -r requirements.txt
python run_all.py --no-install
python audit/release_metrics.py
```

Build the dynamic-server UI before running the complete integration suite. In PowerShell:

```
$env:VNEDU_SITE_DIR = 'site_server'
$env:VNEDU_SITE_ROOT = '/'
python scripts/step7_publish.py
python -m pytest tests -q
```

Unset the two UI environment variables before building the GitHub Pages variant. The default pipeline already builds that variant. Existing output directories must have the publisher's generated-site markers.

From docs/report, compile the English edition with an installed TeX distribution:

```
latexmk -lualatex -interaction=nonstopmode -halt-on-error main-en.tex
```

The PDF uses Libertinus and Noto Sans Mono. The numeric macros come from `tables/audit-metrics.tex`; regenerate them after data or ontology changes. `data/reports/audit-metrics.json` contains the corresponding machine-readable values. The original Vietnamese source and PDF remain available as historical project artifacts.

B A competency-query example

This query counts classified institutions by current province using the retained geographic inference. It excludes institutions explicitly marked as defunct; the absence of that marker still does not prove current operation.

```
PREFIX vnedu: <https://pham-ng.github.io/Vietnam-University-Knowledge-Graph-ver2/ontology#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
SELECT ?province (COUNT(DISTINCT ?institution) AS ?count)
WHERE {
  ?institution a vnedu:HigherEducationInstitution ;
               vnedu:locatedIn ?p .
  ?p a vnedu:Province ; rdfs:label ?province .
  FILTER(LANG(?province) = "vi")
  FILTER NOT EXISTS { ?institution a vnedu:DefunctInstitution }
}
GROUP BY ?province
ORDER BY DESC(?count)
```

The example illustrates an essential reporting distinction: a query can execute correctly against the model while its answer remains dependent on the completeness and meaning of the underlying evidence.

References

- [1] R. Cyganiak, D. Wood, and M. Lanthaler, eds. *RDF 1.1 Concepts and Abstract Syntax*. W3C Recommendation, 2014. <https://www.w3.org/TR/rdf11-concepts/>.
- [2] B. Motik et al., eds. *OWL 2 Web Ontology Language Profiles*, second edition. W3C Recommendation, 2012. <https://www.w3.org/TR/owl2-profiles/>.
- [3] H. Knublauch and D. Kontokostas, eds. *Shapes Constraint Language (SHACL)*. W3C Recommendation, 2017. <https://www.w3.org/TR/shacl/>.
- [4] S. Harris and A. Seaborne, eds. *SPARQL 1.1 Query Language*. W3C Recommendation, 2013. <https://www.w3.org/TR/sparql11-query/>.
- [5] T. Lebo, S. Sahoo, and D. McGuinness, eds. *PROV-O: The PROV Ontology*. W3C Recommendation, 2013. <https://www.w3.org/TR/prov-o/>.
- [6] L. Sauermann and R. Cyganiak, eds. *Cool URIs for the Semantic Web*. W3C Interest Group Note, 2008. <https://www.w3.org/TR/cooluris/>.
- [7] T. Berners-Lee. *Linked Data*, including the five-star scheme. Design Issues, 2006, scheme added 2010. <https://www.w3.org/DesignIssues/LinkedData.html>.
- [8] Silk Framework contributors. *Silk Framework 3.6.0: Link Specification Language and implementation*. <https://github.com/silk-framework/silk/tree/v3.6.0>. Accessed 7 October 2026.
- [9] Apache Jena. *Apache Jena Fuseki*. Project documentation, accessed 7 October 2026. <https://jena.apache.org/documentation/fuseki2/>.
- [10] Apache Jena. *Fuseki HTTP Administration Protocol*. <https://jena.apache.org/documentation/fuseki2/fuseki-server-protocol.html>.
- [11] C. Ogbuji, ed. *SPARQL 1.1 Graph Store HTTP Protocol*. W3C Recommendation, 2013. <https://www.w3.org/TR/sparql11-http-rdf-update/>.
- [12] Apache Jena. *Security in Fuseki2*. <https://jena.apache.org/documentation/fuseki2/fuseki-security.html>.
- [13] Schema.org. *parentOrganization; employee*. Vocabulary definitions, accessed 7 October 2026. <https://schema.org/parentOrganization>; <https://schema.org/employee>.